

Angular: creación de componentes y directivas

Componentes standalone, templates, bindings y control de vistas

Sandra Cano

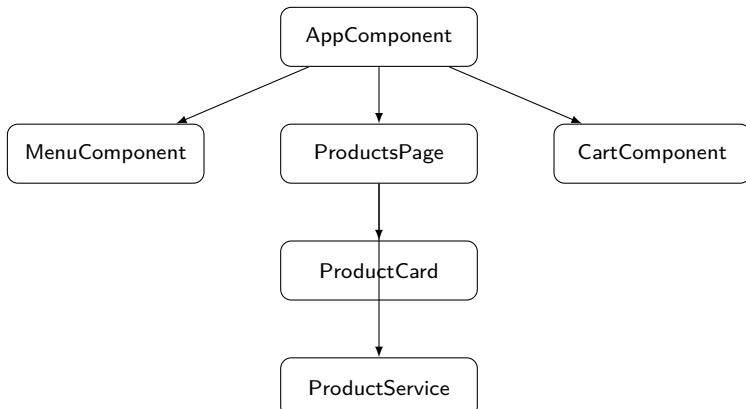
Pontificia Universidad Católica de Valparaíso
OII436-1 Ingeniería Web Avanzada

Segundo semestre de 2026

Angular como arquitectura basada en componentes

Idea principal

Angular organiza el frontend como un conjunto de **componentes reutilizables**, cada uno con una responsabilidad de interfaz, comportamiento y presentación.



¿Qué es un componente?

Componente

Un componente combina una **clase TypeScript**, un **template HTML** y, opcionalmente, estilos CSS.



- La clase define datos y comportamiento.
- El template muestra los datos y captura eventos.
- Los estilos definen apariencia visual.

Crear un componente con Angular CLI

Comando

Angular CLI permite generar automáticamente los archivos de un componente.

```
ng generate component components/product-card
```

Forma abreviada:

```
ng g c components/product-card
```

Archivos generados

```
src/app/components/product-card/  
|-- product-card.ts  
|-- product-card.html  
|-- product-card.css  
'-- product-card.spec.ts
```

Modelo de datos: interfaz Product

Interfaz

Una interfaz define la forma esperada de los datos que usará la aplicación.

```
export interface Product {  
  id: number;  
  nombre: string;  
  description: string;  
  images: string[];  
  price: number;  
  stock: number;  
}
```

Precisión

`images: string[]` indica una lista de textos, por ejemplo URLs o rutas de imágenes.

Componente con lista de productos

```
import { Component } from '@angular/core';
import { Product } from '../models/product';

@Component({
  selector: 'app-products-page',
  imports: [],
  templateUrl: './products-page.html',
})
export class ProductsPage {
  productos: Product[] = [
    {
      id: 1,
      nombre: 'Notebook Lenovo',
      description: 'Notebook de 14 pulgadas',
      images: ['assets/notebook.jpg'],
      price: 699990,
      stock: 8
    }
  ];
}
```

Mostrar productos en el HTML con ul y li

```
<ul>
  @for (producto of productos; track producto.id) {
    <li>
      <strong>{{ producto.nombre }}</strong>
      <br>
      {{ producto.description }}
      <br>
      Precio: ${{ producto.price }}
      <br>
      Stock: {{ producto.stock }}
    </li>
  }
</ul>
```

¿Qué ocurre?

@for recorre el arreglo productos y genera un li por cada producto.

Track

track producto.id ayuda a Angular a identificar cada elemento cuando la lista cambia.

Bindings en Angular

Tipo	Uso	Ejemplo
Interpolación	Mostrar texto dinámico	<code>producto.nombre</code>
Property binding	Asignar una propiedad HTML	<code>[src]=imagen"</code>
Event binding	Responder a eventos del usuario	<code>(click)=agregar()"</code>
Two-way binding	Sincronizar entrada y variable	<code>[(ngModel)]=nombre"</code>

Idea clave

El template no es HTML estático: Angular permite conectar la vista con los datos y métodos definidos en la clase TypeScript.

Ejemplo con imagen y evento

```
<ul>
  @for (producto of productos; track producto.id) {
    <li>
      <img [src]="producto.images[0]" [alt]="producto.nombre" width="120">

      <h3>{{ producto.nombre }}</h3>
      <p>{{ producto.description }}</p>
      <p>Precio: ${{ producto.price }}</p>

      <button (click)="agregarAlCarrito(producto)">
        Agregar al carrito
      </button>
    </li>
  }
</ul>
```

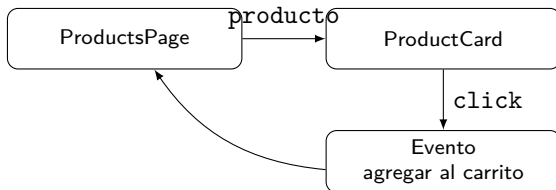
Idea clave

[src] asigna una propiedad de forma dinámica y (click) ejecuta un método cuando el usuario presiona el botón.

Comunicación entre componentes

Escenario

Una página puede tener una lista de productos y delegar la presentación de cada producto a un componente reutilizable.



- El componente padre entrega datos al hijo.
- El componente hijo puede emitir eventos hacia el padre.

Entrada de datos con @Input

```
import { Component, Input } from '@angular/core';  
import { Product } from '../models/product';
```

```
@Component({  
  selector: 'app-product-card',  
  imports: [],  
  templateUrl: './product-card.html'  
})  
export class ProductCard {  
  @Input() producto!: Product;  
}
```

```
@for (producto of productos; track producto.id) {  
  <app-product-card [producto]="producto"></app-product-card>  
}
```

Idea clave

@Input permite que un componente reciba datos desde su componente padre.

¿Qué es una directiva?

Directiva

Una directiva permite agregar o modificar comportamiento en elementos del DOM, componentes o fragmentos del template.

Tipo	Función
Directiva de atributo	Modifica apariencia o comportamiento de un elemento.
Directiva estructural	Crea, elimina o repite fragmentos del template.
Directiva de navegación	Permite navegar entre rutas, por ejemplo <code>routerLink</code> .

Idea clave

Los componentes crean piezas de interfaz; las directivas modifican cómo se comportan o muestran esas piezas.

Directivas de atributo: ngClass y ngStyle

```
<li [ngClass]="{ 'sin-stock': producto.stock === 0 }">
  {{ producto.nombre }}
</li>
```

```
<p [ngStyle]="{ color: producto.stock > 0 ? 'green' : 'red' }">
  Stock: {{ producto.stock }}
</p>
```

Importante en standalone

Si se usan directivas como NgClass o NgStyle, deben importarse en el componente o mediante CommonModule.

Control de flujo actual: @if y @for

```
@for (producto of productos; track producto.id) {  
  <li>  
    <strong>{{ producto.nombre }}</strong>  
  
    @if (producto.stock > 0) {  
      <span>Disponible</span>  
    } @else {  
      <span>Sin stock</span>  
    }  
  </li>  
}
```

Angular actual

Para condiciones y repeticiones comunes se utiliza la sintaxis de control flow: @if, @for y @switch.

Navegación con routerLink

RouterLink

`routerLink` es una directiva que permite navegar entre rutas de Angular sin recargar completamente la página.

```
<nav>
  <a routerLink="/">Inicio</a>
  <a routerLink="/productos">Productos</a>
</nav>

<router-outlet></router-outlet>
```

Relación

`routerLink` indica a qué ruta navegar y `router-outlet` indica dónde mostrar el componente asociado.

Ejemplo de rutas

```
import { Routes } from '@angular/router';
import { Home } from '../pages/home/home';
import { ProductsPage } from '../pages/products-page/products-page';

export const routes: Routes = [
  { path: '', component: Home },
  { path: 'productos', component: ProductsPage }
];
```

routerLink="/productos" → Angular Router → ProductsPage → router-outlet

Directiva personalizada simple

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appResaltar]'
})
export class ResaltarDirective {
  constructor(private element: ElementRef) {
    this.element.nativeElement.style.backgroundColor = '#fff3cd';
  }
}
```

```
<p appResaltar>
  Producto destacado
</p>
```

Idea

Una directiva personalizada permite encapsular comportamiento reutilizable para aplicarlo a distintos elementos.

Actividad práctica

Desafío

Construir una lista de productos utilizando componentes y directivas.

- 1 Crear la interfaz `Product`.
- 2 Crear un componente `ProductsPage`.
- 3 Definir un arreglo productos: `Product[]`.
- 4 Mostrar la lista con `ul`, `li` y `@for`.
- 5 Mostrar Disponible o Sin stock con `@if`.
- 6 Agregar navegación con `routerLink` hacia `/productos`.

Evidencia esperada

La aplicación debe mostrar al menos tres productos y diferenciar visualmente los productos sin stock.

Resumen de la sesión

Concepto	Idea central
Componente	Unidad reutilizable de interfaz con clase, template y estilos.
Standalone	El componente declara sus propias dependencias en <code>imports</code> .
Binding	Conecta datos y eventos entre clase TypeScript y template.
@for	Repite contenido a partir de una colección.
@if	Muestra contenido según una condición.
Directiva	Agrega o modifica comportamiento en el template.
routerLink	Navega entre rutas internas de Angular.

Idea final

Angular permite construir interfaces mantenibles combinando componentes pequeños, templates dinámicos y directivas reutilizables.